

Báo Cáo

Tác vụ 1: Giải thích khái niệm phức tạp (Toán học - Trị riêng)

Prompt cơ bản: "Giải thích cho tôi về giá trị riêng (eigenvalues) là gì?"

Giá trị riêng là một khái niệm trong đại số tuyến tính dùng để mô tả cách một phép biến đổi tuyến tính tác động lên những hướng đặc biệt trong không gian.

Nói dễ hiểu hơn: khi một ma trận biến đổi một vector, thường vector đó sẽ đổi cả hướng lẫn độ dài. Nhưng có một số vector đặc biệt mà sau biến đổi, chúng vẫn giữ nguyên phương, chỉ bị phóng to, thu nhỏ, hoặc đổi chiều. Mức độ phóng to hay thu nhỏ đó chính là giá trị riêng.

Ví vậy, giá trị riêng cho biết:

- một hướng nào đó có được giữ nguyên sau biến đổi hay không
- nếu có, thì vector trên hướng đó bị co giãn bao nhiêu lần

Nếu giá trị riêng dương, vector giữ nguyên chiều.
Nếu giá trị riêng âm, vector bị đảo chiều.
Nếu giá trị riêng có độ lớn lớn hơn 1, vector bị kéo dài.
Nếu độ lớn nhỏ hơn 1, vector bị thu ngắn.

Tóm lại, giá trị riêng là hệ số cho biết một phép biến đổi tuyến tính làm thay đổi độ lớn của các vector ở những hướng đặc biệt như thế nào, trong khi vẫn giữ chúng trên cùng một phương.

Prompt cải tiến: "Hãy giải thích khái niệm giá trị riêng trong đại số tuyến tính cho sinh viên năm nhất. Sử dụng ngôn ngữ dễ hiểu và đưa ra một ví dụ minh họa cụ thể."

Giá trị riêng (eigenvalue) là một khái niệm trong đại số tuyến tính dùng để mô tả cách một ma trận (hay phép biến đổi tuyến tính) tác động lên các vector.

Ý tưởng đơn giản

Khi bạn nhân một ma trận với một vector, thường thì vector đó sẽ:

- bị đổi hướng
- và đổi độ dài

Tuy nhiên, có một số vector đặc biệt mà sau khi biến đổi:

- không đổi hướng
- chỉ bị kéo dài, thu ngắn, hoặc đảo chiều

Những vector này gọi là **vector riêng**, còn số cho biết mức độ co giãn gọi là **giá trị riêng**.

🌟 **Nói ngắn gọn:**
Giá trị riêng cho biết một vector đặc biệt bị "phóng to hay thu nhỏ bao nhiêu lần" sau khi qua phép biến đổi.

Ví dụ minh họa

Xét ma trận:

$$A = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix}$$

Lấy vector:

$$v = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Khi nhân:

$$Av = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

Ta thấy:

- hướng không đổi (vẫn nằm trên trục x)
- độ dài gấp 2 lần

→ Vậy:

- v là vector riêng
- giá trị riêng tương ứng là 2

Prompt nâng cao (Role + CoT): "Bạn là một giảng viên toán học tại UET. Hãy giải thích khái niệm Giá trị riêng (\$eigenvalues\$) và Vector riêng (\$eigenvectors\$) bằng phương pháp Chain-of-Thought."

Mình sẽ giải thích theo từng bước rõ ràng, thay vì trình bày suy nghĩ nội bộ chi tiết.

Bước 1: Ý nghĩa hình học

Hãy xem ma trận A như một phép biến đổi không gian: nó có thể quay, kéo dãn, nén, hoặc lật các vector.

Thông thường, khi một vector đi qua phép biến đổi này, nó sẽ đổi cả hướng lẫn độ dài. Nhưng có một số hướng đặc biệt mà vector nằm trên đó sau biến đổi vẫn giữ nguyên phương, chỉ thay đổi độ dài hoặc đổi chiều.

- Những vector nằm trên các hướng đặc biệt đó gọi là **vector riêng**.
- Mức độ kéo dãn hoặc thu nhỏ trên hướng đó gọi là **giá trị riêng**.

Hiểu trực giác:

- Nếu giá trị riêng là 2, vector bị kéo dài gấp đôi.
- Nếu là 0.5, vector bị thu ngắn còn một nửa.
- Nếu là -1, vector giữ nguyên độ dài nhưng bị đảo chiều.

Vì vậy, vector riêng cho biết hướng đặc biệt, còn giá trị riêng cho biết mức biến đổi trên hướng đó.

Bước 2: Công thức toán học

Khái niệm này được viết bằng công thức:

$$Av = \lambda v$$

Trong đó:

- A là ma trận hoặc phép biến đổi tuyến tính
- v là vector riêng ($v \neq 0$)
- λ là giá trị riêng

Ý nghĩa của công thức này là:

Khi nhân ma trận A với vector v , kết quả không bị lệch sang hướng khác, mà chỉ bằng chính vector đó nhân với một số λ .

Nói cách khác:

- v giữ nguyên phương
- λ cho biết co giãn bao nhiêu lần

-So sánh 3 prompt

Tiêu chí	Prompt 1 (cơ bản)	Prompt 2 (cải tiến)	Prompt 3 (nâng cao)
Mục tiêu	Chung chung	Rõ ràng (cho SV năm nhất)	Rất rõ + định hướng học thuật
Mức độ chi tiết	Thấp	Trung bình	Cao
Yêu cầu cụ thể	Không có	Có (dễ hiểu + ví dụ)	Có (role + 3 bước + ứng dụng)
Cấu trúc	Không rõ	Tương đối rõ	Rất rõ, chia bước logic
Chất lượng câu trả lời	Dễ lan man	Dễ hiểu, có minh họa	Logic, hệ thống, đầy đủ
Tính kiểm soát output	Thấp	Trung bình	Cao

Tác vụ 2: Tạo bộ câu hỏi ôn tập (Lập trình Java)

Prompt cơ bản: "Tạo cho tôi 5 câu hỏi về lập trình hướng đối tượng (OOP) trong Java."

Dưới đây là 5 câu hỏi cơ bản về lập trình hướng đối tượng (OOP) trong Java:

1. Lập trình hướng đối tượng (OOP) là gì và nó khác gì so với lập trình hướng thủ tục?
2. Trong Java, class và object khác nhau như thế nào?
3. Tính đóng gói (encapsulation) là gì? Cho ví dụ trong Java.
4. Kế thừa (inheritance) trong Java hoạt động như thế nào?
5. Tính đa hình (polymorphism) là gì và có những dạng nào trong Java?

Prompt cải tiến: "Hãy tạo 5 câu hỏi trắc nghiệm về tính Đa hình (Polymorphism) và Đóng gói (Encapsulation) trong Java. Mỗi câu hỏi cần có 4 lựa chọn và đáp án đi kèm."

Dưới đây là 5 câu hỏi trắc nghiệm về Polymorphism và Encapsulation trong Java:

Câu 1: Tính đóng gói (Encapsulation) trong Java nhằm mục đích gì?

- A. Tăng tốc độ chương trình
- B. Che giấu dữ liệu và bảo vệ trạng thái đối tượng
- C. Tăng số lượng class
- D. Cho phép kế thừa nhiều class

👉 Đáp án: B

Câu 2: Từ khóa nào thường được sử dụng để thực hiện đóng gói trong Java?

- A. public
- B. protected
- C. private
- D. static

👉 Đáp án: C

Câu 3: Đa hình (Polymorphism) trong Java cho phép điều gì?

- A. Một biến chỉ có thể lưu một kiểu dữ liệu
- B. Một phương thức có nhiều cách thực thi khác nhau
- C. Một class có nhiều constructor
- D. Một chương trình có nhiều package

👉 Đáp án: B

Câu 4: Phương thức override (ghi đè) thuộc loại đa hình nào?

- A. Compile-time polymorphism
- B. Runtime polymorphism
- C. Static polymorphism
- D. Encapsulation

👉 Đáp án: B

Câu 5: Getter và Setter trong Java thường được dùng để làm gì?

- A. Tăng tốc độ xử lý
- B. Truy cập trực tiếp biến private
- C. Kiểm soát việc truy cập và thay đổi dữ liệu
- D. Tạo object mới

👉 Đáp án: C

Prompt nâng cao (Few-shot + Context): "Bạn là một chuyên gia phỏng vấn kỹ thuật tại Google. Hãy tạo bộ câu hỏi ôn tập về Java OOP cho sinh viên đang làm đồ án Client-Server."

Câu 1:
Trong đoạn code sau, điều gì sẽ xảy ra?

Code snippet:

```
Java
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Test {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();
    }
}
```

Giải thích:
Đây là ví dụ về runtime polymorphism (đa hình lúc chạy). Biến `a` có kiểu `Animal` nhưng trỏ đến đối tượng `Dog`, nên phương thức `sound()` của `Dog` sẽ được gọi.

👉 Kết quả: "Dog barks"

Lỗi thường gặp:
Sinh viên hay nghĩ rằng sẽ gọi `Animal.sound()` vì kiểu biến là `Animal`, nhưng thực tế Java quyết định dựa trên kiểu object thật.

Câu 2:

Đoạn code sau có lỗi gì?

Code snippet:

Java

```
interface Flyable {
    void fly() {
        System.out.println("Flying");
    }
}
```

Giải thích:

Trong Java (trước Java 8), interface không được phép có method có body (trừ khi dùng `default` hoặc `static` từ Java 8 trở đi).

⚡ Lỗi: method `fly()` có thân nhưng không khai báo `default` hoặc `static`

✓ Cách sửa:

Java

```
default void fly() {
    System.out.println("Flying");
}
```

Lỗi thường gặp:

Nhầm lẫn giữa abstract class và interface, đặc biệt khi viết method có thân.

Câu 3:

Đoạn code sau có vấn đề gì?

Code snippet:

Java

```
interface A {
    void show();
}

interface B {
    void show();
}

class Test implements A, B {
    public void show() {
        System.out.println("Hello");
    }
}
```

Giải thích:

Đoạn code này không lỗi vì cả hai interface đều có cùng phương thức `show()` với cùng signature, nên class `Test` chỉ cần implement một lần.

-So sánh:

Tiêu chí	Prompt cơ bản	Prompt cải tiến	Prompt nâng cao
Mục tiêu	Tạo câu hỏi chung	Tạo câu hỏi trắc nghiệm cụ thể	Tạo bộ câu hỏi theo ngữ cảnh phỏng vấn
Ngữ cảnh	Không có	Có giới hạn nội dung (Polymorphism, Encapsulation)	Rõ ràng (Google interviewer + đề án client-server)
Định dạng	Không yêu cầu	Có (4 lựa chọn + đáp án)	Rất rõ (câu hỏi + code + giải thích)
Mức độ chi tiết	Thấp	Trung bình	Cao
Độ khó	Cơ bản	Trung bình	Trung–cao (gần thực tế)
Tính thực tiễn	Thấp	Trung bình	Cao (gắn với lỗi thực tế, interview)
Khả năng kiểm soát output	Thấp	Khá	Rất cao

Tác vụ 3: Tóm tắt tài liệu học thuật (Tiếng Anh chuyên ngành)

Đoạn tiếng Anh:

Artificial intelligence is transforming many industries, especially healthcare and education. In healthcare, AI helps doctors detect diseases earlier and improve treatment plans. In education, it supports personalized learning experiences for students.

Tóm tắt bằng tiếng Việt:

Trí tuệ nhân tạo đang làm thay đổi nhiều lĩnh vực, đặc biệt là y tế và giáo dục. Trong y tế, AI giúp phát hiện bệnh sớm và hỗ trợ điều trị tốt hơn. Trong giáo dục, AI giúp cá nhân hóa việc học cho học sinh.

Prompt cơ bản: "Tóm tắt đoạn văn bản sau bằng tiếng Việt."

Prompt cải tiến: "Hãy tóm tắt tài liệu học thuật dưới đây thành một danh sách khoảng 3 ý chính. Tập trung vào các kết luận quan trọng nhất."

- Trí tuệ nhân tạo đang thay đổi mạnh mẽ nhiều lĩnh vực, đặc biệt là y tế và giáo dục.
- Trong y tế, AI giúp phát hiện bệnh sớm và cải thiện kế hoạch điều trị.
- Trong giáo dục, AI hỗ trợ cá nhân hóa trải nghiệm học tập cho người học.

Prompt nâng cao (Structured Output): "Bạn là một trợ lý nghiên cứu học thuật. Hãy đọc tài liệu về 'Ứng dụng Blockchain trong bảo mật' sau đây và tóm tắt theo cấu trúc:

Đoạn tiếng Anh:

Blockchain technology is increasingly used in cybersecurity to enhance data integrity and prevent unauthorized access. This study analyzes various blockchain-based security models, including decentralized identity management and secure data sharing systems. The research applies comparative analysis and case studies to evaluate performance. Results show that blockchain improves transparency, reduces risks of data tampering, and enhances trust in distributed systems. However, challenges such as scalability, high energy consumption, and implementation complexity remain significant barriers.

Mục đích nghiên cứu:

Nghiên cứu ứng dụng của blockchain trong việc tăng cường bảo mật dữ liệu và ngăn chặn truy cập trái phép.

Phương pháp luận:

- Phân tích so sánh (comparative analysis) các mô hình bảo mật dựa trên blockchain
- Nghiên cứu tình huống (case studies) về các hệ thống thực tế
- Đánh giá hiệu năng của các giải pháp như decentralized identity management và secure data sharing

Kết quả then chốt:

- Blockchain cải thiện tính toàn vẹn dữ liệu (data integrity)
- Giảm nguy cơ bị thay đổi dữ liệu (data tampering)
- Tăng tính minh bạch (transparency) và độ tin cậy trong hệ thống phân tán (distributed systems)

Hạn chế của tài liệu:

- Khả năng mở rộng còn hạn chế (scalability)
- Tiêu thụ năng lượng cao (high energy consumption)
- Độ phức tạp trong triển khai (implementation complexity)

-So sánh:

Tiêu chí	Prompt cơ bản	Prompt cải tiến	Prompt nâng cao
Mục tiêu	Tóm tắt đơn giản	Tóm tắt thành các ý chính	Tóm tắt có cấu trúc học thuật
Định dạng output	Đoạn văn	Danh sách bullet	Cấu trúc rõ (mục đích, phương pháp, kết quả, hạn chế)
Mức độ chi tiết	Thấp	Trung bình	Cao
Yêu cầu cụ thể	Không có	Có (số ý, tập trung kết luận)	Rất cụ thể (nhiều mục + ngôn ngữ + thuật ngữ)
Tính học thuật	Thấp	Trung bình	Cao
Khả năng tái sử dụng	Thấp	Khá	Rất cao (dùng như template nghiên cứu)
Khả năng kiểm soát output	Thấp	Trung bình	Rất cao

Prompt nâng cao hiệu quả hơn vì :

1. **Context (Ngữ cảnh):** Việc gán vai trò (Role prompting) giúp AI xác định được "tông giọng" và độ sâu của kiến thức cần cung cấp.
2. **Constraints (Ràng buộc):** Khi bạn giới hạn định dạng (ví dụ: "tóm tắt trong 5 ý"), AI sẽ lọc bỏ thông tin thừa.
3. **Chain-of-Thought (Chuỗi tư duy):** Yêu cầu AI giải thích từng bước giúp giảm thiểu tình trạng "hallucination" (AI nói sai một cách tự tin), đặc biệt quan trọng với các môn Toán hay Lý.

Tổng hợp các nguyên tắc và mẹo viết prompt hiệu quả dựa trên kết quả thử nghiệm:

+) **Công thức 4 thành phần:** **Vai trò** (Bạn là...) + **Nhiệm vụ** (Hãy làm...) + **Bối cảnh** (Tôi đang học...) + **Định dạng** (Trả lời dưới dạng...).

+) **Cung cấp ví dụ (Few-shot):** Nếu muốn AI trả lời theo ý mình, hãy cho nó 1-2 ví dụ mẫu về câu trả lời bạn mong muốn.

+) **Tương tác lặp (Iterative):** Đừng mong đợi prompt đầu tiên hoàn hảo. Hãy sử dụng kết quả của prompt trước để điều chỉnh prompt sau (ví dụ: "Giải thích lại bước 2 nhưng kỹ hơn").